

The background of the cover features abstract, wavy green lines that create a sense of movement and depth. These lines are concentrated in the corners and bottom, leaving the central area where the text is located relatively clear.

AUTOMATION TESTING PORTFOLIO

Prepared By :

MOHD ZAMIR BIN MUHAMAD YUSOF

QA Professional – Manual
Testing with Automation
Experience

INTRODUCTION

I am Zamir Yusof, an ISTQB-certified QA professional with over 13 years of experience, combining expert manual testing with practical automation skills. My automation work has focused on strengthening regression coverage, improving efficiency, and solving complex challenges such as OTP handling, dynamic test data generation, and locator management.

At the same time, I bring a strong foundation in manual QA – ensuring functional accuracy, regression reliability, and UAT readiness across ERP, telecom, and e-commerce systems. By integrating automation thoughtfully, I have helped teams achieve more scalable, maintainable, and faster testing outcomes while keeping business workflows at the center of quality assurance.

This portfolio demonstrates not only my technical contributions but also how I bridge manual and automation testing to deliver solutions that are practical, business-aligned, and impactful for both technical and non-technical stakeholders.

AUTOMATION TEST CASE SELECTION

Automation testing is often described as the future of software quality assurance, but in practice it requires careful planning and the right conditions to succeed. Not every test case is suitable for automation. Before automating, several key criteria must be met:

- **System stability** – The System Under Test (SUT) should be relatively stable. Attempting automation during early phases such as SIT, when bugs and UI changes are frequent, often leads to wasted effort. Automation is most effective after UAT or during regression testing, where hundreds of cases can be executed reliably.
- **Automation-friendly design** – UI elements must be identifiable through unique locators. Web applications typically allow concise XPath locators (e.g., `//b[text()='Total Free Unit']`).
 - Mobile applications, however, may rely on buttons or images without identifiers, requiring collaboration with developers to add unique IDs or fallback indexing methods (e.g., `(//android.view.ViewGroup[@index='2'])[3]//android.widget.TextView[@text='1']`).
- **Minimal dependency on complex tasks** – Features such as OTP login introduce complexity, as codes are often sourced from external apps. Where OTP cannot be disabled in test environments, I have implemented custom solutions, such as integrating Java programs to generate TOTP codes directly within the automation framework.
- **Easily prepared test data** – Accurate and accessible test data is critical. For example, in postpaid line registration, NRIC numbers must be unique and aligned with MSISDN availability. I developed scripts to generate randomized digits while allowing predefined values, ensuring valid test data pools.
- **Machine-interpretable results** – Automation requires outcomes that can be validated programmatically. While image quality cannot be reliably tested, automation can confirm success messages, element existence, or specific file names to verify results.

EXAMPLE – MINIMAL DEPENDENCY ON COMPLEX TASK

Automation is most effective when test cases do not rely heavily on complex or external processes. A common example is user login with OTP verification.

While simple for a human tester, OTP introduces significant challenges in automation because the code is often sourced from external applications such as Google Authenticator or email. This requires the automation engineer to not only test the System Under Test (SUT) but also replicate interactions with third-party systems.

In most projects, OTP is disabled in the test environment to simplify execution. However, in one project I worked on, OTP could not be bypassed. To address this, I implemented a custom solution:

- Developed a Java program to generate TOTP codes within the test automation system, eliminating reliance on external apps.
- Integrated OTP generation into the automation scripts, allowing the framework to fetch and input valid codes during login.
- Ensured seamless execution of login flows, enabling automated regression testing even in environments where OTP was mandatory.

```
1 package com.born;
2
3 import java.util.Properties;
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19 public class Authenticator {
20
21     /* Perform steps below for 1st time setup
22     * 1. Navigate to https://xxxx.microsoft.com/security-info
23     * 2. Add sign-in method
24     * 3. Choose method Authenticator app
25     * 4. Choose "I want to use different app"
26     * 5. Proceed until asked to scan QR code, click "Can't scan image"
27     * 6. Copy both account name and secret key to related variables below then save this file.
28     * 7. Back to MFA setup page, click Next to enter 6 digit code
29     * 8. Run this program to generate the 6 digit code and complete the MFA setup using the same.
30     */
31
32     public static String otpcode;
33     public static String MFAaccount = "xxxxx:V1S705X@xxxx.com.my"; //user's email
34     public static String secretkey = "2pgrlcryxxxxx"; //user's secret key
35
36
37
38     public static void main(String[] args) {
39         otpcode = getTOTPCode();
40         System.out.println(otpcode);
41         try {
42             Thread.sleep(2000);
43         } catch (InterruptedException e) {
44             System.out.println("got interrupted!");
45         }
46     }
47     public static String getTOTPCode() {
48
49         Totp totp = new Totp(secretkey);
50         String twoFactorCode = totp.now();
51         return twoFactorCode;
52     }
53 }
```

```
public String getOTPCode (String object, String data) {
    APP_LOGS.debug("Fetching and entering OTP code " + object);
    try {

        driver.findElement(By.xpath(OR.getProperty(object))).click();
        System.out.println("Generating OTP for account " + Authenticator.MFAaccount);
        String gencode = Authenticator.getTOTPCode();
        Thread.sleep(2000);
        System.out.println("OTP code is " + gencode);
        driver.findElement(By.xpath(OR.getProperty(object))).sendKeys(gencode);

    }
    catch (Exception e) {
        return Constants.KEYWORD_FAIL + "Unable to get code";
    }
    return Constants.KEYWORD_PASS;
}
```

*screenshot from BORN Group's in-house test automation framework

TEST DATA PREPARATION

Test data is a critical but often overlooked aspect of automation, especially by engineers who have not been deeply involved in manual testing. For automation to be effective, test data must be both accurate and easy to generate. The complexity of the test case often dictates how carefully data needs to be prepared.

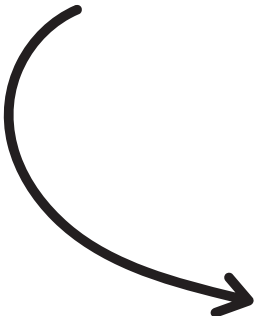
Example – Postpaid Line Registration:

During new line registration, users must provide a unique NRIC that has not been registered, while also matching the availability of MSISDN numbers based on NRIC patterns. For instance, if the last four digits of the NRIC are 1234, the system will suggest MSISDNs ending with 1234.

To support this requirement, I developed methods and scripts that:

- **Generated randomized NRIC digits** while allowing predefined values for controlled scenarios.
- **Validated MSISDN availability** against NRIC patterns to ensure test data consistency.
- Automated the entire data entry process, reducing manual preparation time and ensuring repeatability across regression cycles.

```
public void randomNumber() throws Exception
{
    String num = keyRefPar.getCurrentInputData_Pays();
    By byobjectname= uimap.getStoreFinder(keyRefPar.getCurrentObjectName());
    try {
        Random random = new Random();
        String randNumber = String.format("%0"+num+"d", random.nextInt(10000));
        driver.findElement(byobjectname).sendKeys(randNumber);
        report.updateTestLog("Generate" +num+ "digits Random Number"+keyRefPar.getCurrentObjectName(),
            "Random Number" +randNumber+ "set successfully ",Status.PASS);
    }catch (Exception e) {
        report.updateTestLog("Generate" +num+ "digits Random Number"+keyRefPar.getCurrentObjectName(),
            "Random Number Failed ",Status.FAIL);
    }
}
```



Action	LabelName	ObjectName	TestDataReference_Pays	InputData_Pays
comment	Step 2 Navigate to URL defined in the test case			
goToURL				URL_ATLNLPD_TC01
comment	Step 3 Fill Personal Details and proceed			
click	Select NRIC	txtNL_NricFld1		
setText	Enter NRIC	txtNL_NricFld1		<NRIC1>
randomnumber	Enter NRIC	txtNL_NricFld2		2
setText		txtNL_NricFld3		0
randomnumber	Enter NRIC	txtNL_NricFld3		3
click	Click Check Terms and conditions	chkTnC_EMV	JSScript	
click	Click Proceed button	btnProceed_NRICScr		

*screenshot from BORN Group's test automation Excel sheet

MACHINE-INTERPRETABLE RESULTS

Automation is only effective when outcomes can be reliably interpreted by the system. Unlike manual testing, where a human can visually assess elements such as image quality, automation requires clear, machine-readable indicators of success or failure.

For example, verifying the quality of images displayed on a webpage is not feasible through automation.

However, automation can validate:

- The presence of an image file with a specific filename.
- Success or error messages displayed on the UI.
- Existence of expected elements on a confirmation page.

Example – Line Registration Success Page

In one project, I designed automated scripts to verify:

- The presence of a success message (e.g., “Your order is successful!”).
- The existence of key UI elements confirming registration completion.
- Runtime capture of order IDs and promotional elements for reporting.

This approach ensured that automated tests provided consistent, machine-interpretable validation of outcomes, reducing reliance on manual checks and improving regression efficiency.

Action	LabelName	ObjectName	TestDataReference_Pays	InputData_Pays	ExpectedData_Pays	Opt_Param_Pays
waitForsec						
waittillobjexist		elmOrderConfMsg				
<i>comment</i>	Step 12 Verify order summary page					
verifyelementtextcontains	Verify Order Message	elmOrderConfMsg		Your order is successful!		h1
getObjectTextRunTime		lblOrderConf_OrderID	order			CJ:strOrderID
scrollPage				down		
verifyobjexist		elmAmt_NLTC01_PromoSucc				
<i>comment</i>	End of the test case					

*screenshot from BORN Group’s test automation Excel sheet

WRITING TEST CASES

Being an automation engineer does not eliminate the need to understand manual test execution. In fact, recognizing the **true passing criteria** of a test case is essential. While automation often focuses on achieving the end outcome (e.g., placing an order), my years of manual testing experience have taught me that multiple checkpoints across different pages or flows must be validated to ensure accuracy.

Example – Postpaid Line Registration:

During automation of new line registration, I identified a hidden requirement: **the MSISDN must auto-populate based on NRIC digits**. Because the MSISDN pool in the test environment was limited, NRIC values had to be customized to align with available MSISDNs (e.g., numbers starting with digit 0). By incorporating this logic into the automation scripts, I ensured reliable execution and accurate validation.

In practice, automation test cases are derived from manual ones but expanded with:

- More variations in test data (randomized and predefined values).
- Additional validation points beyond the main outcome.
- Higher volume of cases (often exceeding 100 per cycle) thanks to faster execution.
-

This approach demonstrates how my manual testing expertise directly enhances automation, ensuring that scripts are not only efficient but also comprehensive and reliable.

Action	LabelName	ObjectName	TestDataReference_Pays	InputData_Pays
comment	Step 2 Navigate to URL defined in the test case			
goToURL				URL_ATLNLPD_TC01
comment	Step 3 Fill Personal Details and proceed			
click	Select NRIC	txtNL_NricFld1		
settext	Enter NRIC	txtNL_NricFld1		<NRIC1>
randomnumber	Enter NRIC	txtNL_NricFld2		2
settext		txtNL_NricFld3		0
randomnumber	Enter NRIC	txtNL_NricFld3		3
click	Click Check Terms and conditions	chkTnC_EMV	JSScript	
click	Click Proceed button	btnProceed_NRICScr		

*screenshot from BORN Group's test automation Excel sheet

Example of randomized number,
with fixed 6 initial digits

- 881022-07-0213
- 881022-88-0987
- 881022-12-0012

SELECTING AUTOMATION FRAMEWORK

In my previous employment (BORN Group), I work with two **custom Selenium-based automation frameworks**, each with distinct strengths:

- Framework A – Supports parallel execution but limited to web applications. Less user-friendly when executing the same test case with multiple datasets.
- Framework B – Handles both web and mobile applications in a single script, supports multiple datasets, but does not allow parallel execution.

Like most Selenium-based frameworks, setup requires advanced expertise in automation and programming, including configuration for external test data files, test recording, and reporting. While the R&D team manages core framework development, my contributions include:

- Adding new methods and functionalities based on project requirements.
- Updating existing methods to improve reliability.
- Maintaining locator libraries to ensure scripts remain stable across UI changes.

Working with Specflow

In my current employment at Exact Asia Development Centre (Exact ADC), I am working with SpecFlow, a Behavior-Driven Development (BDD) framework for the .NET ecosystem. SpecFlow uses Gherkin syntax (Given–When–Then) to describe test scenarios in plain English, making them understandable to both technical and non-technical stakeholders. Each step in a scenario is mapped to C# step definitions, which then interact with Selenium or APIs to automate workflows.

SpecFlow is particularly well-suited for Exact Online because the platform is a SaaS solution for accounting, ERP, and business management built heavily on Microsoft technologies. Using SpecFlow provides several advantages:

- Alignment with the .NET stack – Seamless integration with Visual Studio and C#, which are core to Exact Online's development environment.
- Clarity for complex business rules – Finance and ERP processes involve intricate rules; Gherkin scenarios make them easy for non-technical stakeholders (e.g., product owner, customer support) to validate.
- Scalable regression testing – Modules like invoicing, payroll, and inventory benefit from reusable scenarios that can grow with the product.
- Business-driven automation – Ensures test cases reflect real-world workflows, not just technical scripts, bridging the gap between product owners and QA teams.

TEST EXECUTION

During my time at **BORN Group**, I worked with hybrid keyword-data driven Selenium frameworks. In these frameworks, common functionalities (such as clicking elements, capturing text, or verifying objects) were pre-programmed and mapped to specific keywords. Test data was stored externally in Excel sheets, allowing flexible reuse and variation without modifying the core scripts.

This design meant that automation test cases closely resembled manual ones, but with added automation-specific instructions like wait commands and runtime validations. The benefits were clear:

- Consistency – standardized keywords ensured uniform execution across projects.
- Flexibility – externalized test data enabled rapid scaling of test coverage.
- Efficiency – large regression suites could be executed far faster than manual testing.

By leveraging this approach, I expanded regression coverage to over 100 cases per cycle, ensuring comprehensive validation while reducing manual effort.

Action	LabelName	ObjectName	TestDataReference_Pays	InputData_Pays	ExpectedData_Pays	Opt_Param_Pays
waitForSec						
waittillObjectExist		elmOrderConfMsg				
comment	Step 12 Verify order summary page					
verifyelementtextcontains	Verify Order Message	elmOrderConfMsg		Your order is successful!		h1
getObjectTextRunTime		lblOrderConf_OrderID	order			CJ:strOrderID
scrollPage				down		
verifyobjectexist		elmAmt_NLTC01_PromoSucc				
comment	End of the test case					

*screenshot from BORN Group's test automation Excel sheet

Experience with SpecFlow at Exact ADC

In my current role at Exact ADC, I work with SpecFlow, a Behavior-Driven Development (BDD) framework designed for the .NET ecosystem. SpecFlow uses Gherkin syntax (**Given-When-Then**) to describe test scenarios in plain English, making them accessible to both technical and non-technical stakeholders. Each step in a scenario is mapped to C# step definitions, which then interact with Selenium or APIs to automate workflows.

Although SpecFlow differs in format from BORN Group's in-house keyword-driven framework, there are similarities. Both approaches rely on predefined actions for common tasks (e.g., clicking elements, handling dialogs, switching iframes), which reduces the effort required from QA engineers when writing automation tests. The key distinction is that SpecFlow emphasizes business-driven automation: test cases are written to reflect real workflows and requirements, ensuring that automated tests align closely with how end-users interact with the system.

*screenshot from SpecFlow-based test automation at Exact ADC

```
Feature: ShopFloorPlanning
  As a floor manager
  I want to be able to view shop orders requiring planning
  so that I can properly sort shop order operations

  Scenario: Adding fields via customize menu to shop floor planning
    Given I log in as Customer Manufacturing Advanced
    And I export Manufacturing data
    When I add the additional fields via the customize menu
    Then the proper fields and data is shown
```

```
using System;
using System.Diagnostics;
using System.Runtime.CompilerServices;
using Exact.UITest.CodedTests.Framework;
using Exact.UITest.Manufacturing.EOL.PageObjects.Views;

namespace Exact.UITest.Manufacturing.Support
{
    public class BaseSteps
    {
        public void LoginAsCloneUserCustomerManufacturingAdvanced()
        {
            User.LoginCloneUser(TemplateUser.CustomerManufacturingAdvanced);
            Administration.AdminParam("FinancialYearFree", "2012");
            Administration.AdminParam("FinancialYearTo", (DateTime.Now.Year + 1).ToString());
            Administration.NewAdministration(Template.NL.ProductieBedrijfVergevoerd, Legislation.NL);
        }

        public void LoginAsCloneUserCustomerManufacturing()
        {
            User.LoginCloneUser(TemplateUser.CustomerManufacturing);
            Administration.AdminParam("FinancialYearFree", "2012");
            Administration.AdminParam("FinancialYearTo", (DateTime.Now.Year + 1).ToString());
            Administration.NewAdministration("Productie bedrijf (CRM Included)", Legislation.NL);
        }

        public void LoginAsCloneUserCustomerManufacturingPremium()
        {
            User.LoginCloneUser(TemplateUser.CustomerManufacturingPremium);
            Administration.AdminParam("FinancialYearFree", "2012");
        }
    }
}
```

TEST REPORTING AND DEBUGGING

Similar to manual testing, automation requires thorough reporting and debugging at the end of each execution cycle. However, unlike manual testing where defects are typically resolved by developers, failures in automation often stem from issues within the test scripts themselves and must be addressed by the automation engineer.

Example – Locator Changes:

A step such as clicking a home icon may fail if the underlying code or XPath changes. While a human tester can adapt instantly, automation scripts require precise locators. To maintain efficiency, it is not advisable to add optional conditions (e.g., fallback image-based locators with if-else statements), as this increases execution time and script complexity. This reinforces the importance of ensuring the System Under Test (SUT) is stable before automating test cases.

Reporting:

- Automated reports are generated at the end of each run, providing detailed logs of pass/fail status, execution time, and environment details.
- These reports allow quick identification of failures, root cause analysis, and corrective action.
- By maintaining clear reporting standards, I ensured stakeholders had transparent visibility into test outcomes and progress.

This structured approach to reporting and debugging helped maintain high reliability of automation suites, minimized downtime, and ensured that automation remained a trusted complement to manual QA.

*screenshot taken from BORN Group's automated regression test report

SmokeSuite_Regression_MyAccount_TC_19 Automation Execution Results				
Date & Time	2023 0114157	Iteration Mode	RunOneIterationOnly	
Start Iteration	1	End Iteration	1	
Browser	Chrome	Version	54	
Platform	Windows 10	Executed on	cloud.browserstack.com/web/health	
Step No	Step Name	Description	Status	Step Time
+ comment				
1	Test Case Name: Regression_MyAccount_TC_19		DONE	02 Jun 2023 03:14:57
+ comment				
2	Step 1: Navigate to the Home Page		DONE	02 Jun 2023 03:14:58
+ invoke application				
3	Invoke Application	Invoke the URL @ [REDACTED]	PASS	02 Jun 2023 03:14:59
+ delete cookies				
4	Delete all cookies	Deleted all cookies	DONE	02 Jun 2023 03:15:01
+ click				
5	Click the Object	MyMyDialogIcon	PASS	02 Jun 2023 03:15:02
+ set text				
6	Set value in - txt_LoginEmail	value [REDACTED]		
19		elm_AddressBook5 -> Object does not exist =		
+ verify object exist				
20		elmPlus_AddressBook -> Object exist= true As t		
+ comment				
21	Step 3: Proceed to place order and save Sh address			
+ go to URL				
22	Error	null value in entry: url=null		
23	BORN Info	Test case iteration terminated by user Process		
Execution Duration: 7 minutes(s)				
Steps passed: 14 Steps fail: 1				

Sample test report



SummaryTestsTerraform PlanArtifactoryScansCode CoverageAssociated pipelines

Summary

4 Run(s) Completed (3 Passed, 1 Failed) 11 unique failing tests in the last 14 days

Total tests: 31727 (31,078 Passed, 3 Failed, 646 Other)

Failed tests: 3 (3 New, 0 Existing)

Pass percentage: 97.95%

Run duration: 1h 21m

Tests not reported: 0

Bug▼LinkAssociate Test Case

Filter by test or run nameTags<>×

TestDuration

▼ UI tests (3/1845)1:21:44.78

▶ MyFirmUserEditsVATPaymentTerm New0:01:07.59

▶ YasmineAbleToSetPaymentDateAsTodayInBulkAndSeeU0:01:06.77

▶ ReopenProcessedPurchaseInvoice New0:00:23.88

ReopenProcessedPurchaseInvoice

Result Details

✗ Failed Friday on AZDEVOPS-SCALEDDuration0:00:23.887

Ownernot availableDate started4/17/2026, 6:47:13 PM

Date completed4/17/2026, 6:47:37 PMFailing sinceFriday

Failing since build20260417.3

DebugWork itemsAttachmentsHistory

age: Incorrect value for: Account_BankAccountNumberView: expected='NL86 ING8 0002 4455 88', actual='NL84 RAB0 0200 1122 44'

10:47:36

*screenshot from the Azure DevOps automated regression test report at Exact ADC

CONCLUSION

Automation is more than just speeding up test execution, it is about building reliability, reducing risk, and enabling teams to deliver quality software faster. My experience shows how automation can be applied thoughtfully: selecting the right test cases, preparing accurate data, ensuring machine-interpretable results, and maintaining frameworks that evolve with project needs.

By combining my manual QA foundation with automation expertise, I ensure that testing is not only efficient but also comprehensive and aligned with business goals. This portfolio demonstrates my ability to bridge both worlds, delivering solutions that are practical, maintainable, and impactful for organizations seeking to strengthen their quality assurance practices.

KEY ACHIEVEMENTS IN AUTOMATION

- **Solved OTP automation challenges** by developing a custom Java program to generate TOTP codes, enabling login flows in environments where OTP could not be disabled.
- **Enhanced test data preparation** by scripting randomized and predefined NRIC/MSISDN generation, ensuring valid datasets for postpaid line registration.
- **Expanded regression coverage** to 100+ automated cases per cycle, leveraging hybrid keyword-data frameworks for faster execution and broader validation.
- **Maintained and improved in-house Selenium frameworks**, adding methods, updating functionalities, and managing locator libraries to support evolving project needs.
- **Delivered clear, machine-interpretable validations** (success messages, element existence, runtime order IDs), ensuring automation results were reliable and transparent.
- **Produced detailed automated reports** with pass/fail logs, execution times, and environment details, enabling quick debugging and stakeholder visibility.
- **Bridged manual and automation testing expertise**, applying deep knowledge of hidden requirements and validation points to strengthen automation accuracy.